

时长预估方案

一、为什么要单独做时长预估

二、目前有实践价值的方案

三、小红书 EGMN：分布建模路线

四、CREAD 的核心思想

五、样本怎么设计

六、模型怎么设计

七、分类怎么还原成连续时长

八、三个 loss 分别是什么

1. 分类 loss

2. 还原 loss

3. 保序 loss

九、EAD 到底怎么做

Step 1: 准备训练时长分布

Step 2: 生成候选阈值

Step 3: 枚举候选切法

Step 4: 计算还原误差 proxy

Step 5: 计算学习难度 proxy

Step 6: 选择总分最低的阈值方案

十、阈值怎么选：推荐实践

十一、为什么要保持多头单调

十二、长时长头样本稀疏怎么处理

1. head-level pos_weight

2. 长时长样本 oversampling

3. focal loss

4. 两阶段训练

5. 截断和校准

十三、训练代码骨架

十四、离线评估与线上监控

十五、推荐落地路线

第一阶段：建立 baseline

第二阶段：分桶多头

第三阶段：EAD 阈值优化

第四阶段：长时长增强

第五阶段：多目标融合

十六、我建议的第一版配置

十七、总结

参考资料

一、为什么要单独做时长预估

在推荐系统里，点击、进房、互动只能说明用户产生了入口行为，但**观看时长**更接近用户消费质量。

在直播场景里，时长预估尤其重要。用户是否进房只是第一步，真正影响直播间价值的是：

- 1 是否秒退
- 2 是否有效观看
- 3 是否停留超过 1 分钟
- 4 是否形成深度观看
- 5 是否进一步互动、关注、送礼

如果排序系统只优化 CTR，容易把一些“标题吸引但体验一般”的直播间排上来。如果加入时长目标，模型可以更重视真正能让用户停留的直播间。

但时长预估不是一个普通回归问题。它有几个典型难点：

- 1 **1.** 分布极长尾：大部分观看很短，少数观看很久
- 2 **2.** 标签延迟：长观看需要等用户退出或窗口结束才知道
- 3 **3.** 样本稀疏：长时长正样本很少
- 4 **4.** 业务偏置：头部主播、大直播间、流量池会天然影响时长
- 5 **5.** 目标不唯一：减少秒退、提升有效观看、提升长观看并不完全等价

所以工业界通常不会只用一个简单的 MSE 回归头，而会采用加权二分类、分桶多头、CREAD、树式回归、分布建模等方案。

二、目前有实践价值的方案

下面只筛选对工业落地有参考价值的方案。

方案	来源	核心思路	实践评价
直接回归	通用 baseline	直接预测 <code>watch_time</code> 或 <code>log(1+watch_time)</code>	简单，但容易被长尾影响
WLR	YouTube 2016	正样本按观看时长加权，训练二分类，推理用 odds 近似期望时长	工程成本低，适合第一版 baseline
分桶多头	工业常见	预测 <code>P(T > x)</code> 或 <code>P(T in bucket)</code>	稳定、可解释、可监控
CREAD	快手 AAAI 2024	分类 + 还原 + EAD 自适应分桶 + 保序约束	非常适合作为可落地方案
TPM / PTPM	快手 KDD 2023 / 2025	树式渐进回归，建模时长区间层级关系	更复杂，适合后续增强
D2Q / CWM	快手 KDD 2022、KDD 2024	处理视频长度偏置和反事实观看时长	短视频更直接，直播需改造
EGMN	小红书 RecSys 2025	指数-高斯混合分布，直接建模时长分布	理论强，但工程复杂

对于直播观看时长，我建议的优先级是：

- 1 第一版：WLR 或 `log watch_time` 回归做 baseline
 - 2 第二版：`P(T > x)` 分桶多头
 - 3 第三版：CREAD-like，加入还原 loss、保序 loss、EAD 阈值选择
 - 4 第四版：考虑树式回归、分布建模、去偏建模

三、小红书 EGMN：分布建模路线

小红书 RecSys 2025 的 EGMN 全名是：

Multi-Granularity Distribution Modeling for Video Watch Time Prediction via Exponential-Gaussian Mixture Network

它的核心判断是：短视频观看时长不是单一分布，而是多粒度混合分布。

论文观察到两类关键现象：

1. 粗粒度偏态：大量 quick-skip, 时长集中在 0 附近
2. 细粒度多样性：不同用户-视频组合会出现不同观看模式

所以它假设观看时长服从指数分布和多个高斯分布的混合：

$$1 \quad p(t \mid x) = w_0 * \text{Exp}(t \mid \lambda) + \sum w_k * \text{Gaussian}(t \mid \mu_k, \sigma_k^2)$$

其中：

- 1 Exp 部分：拟合快速划走、靠近 0 的偏态分布
- 2 Gaussian 部分：拟合不同用户-视频交互模式下的多峰分布
- 3 w_k ：不同分布成分的混合权重

模型输入用户、视频、上下文特征，经过 backbone 后，预测混合分布参数：

$$1 \quad \lambda, \mu_k, \sigma_k, w_k$$

然后用分布期望作为预测观看时长：

$$1 \quad E[T \mid x] = w_0 * 1/\lambda + \sum w_k * \mu_k$$

训练目标一般包括：

- 1 MLE loss：让真实观看时长在预测分布下概率密度更高
- 2 entropy loss：防止所有样本塌缩到单一分布成分
- 3 regression loss：让分布期望接近真实时长

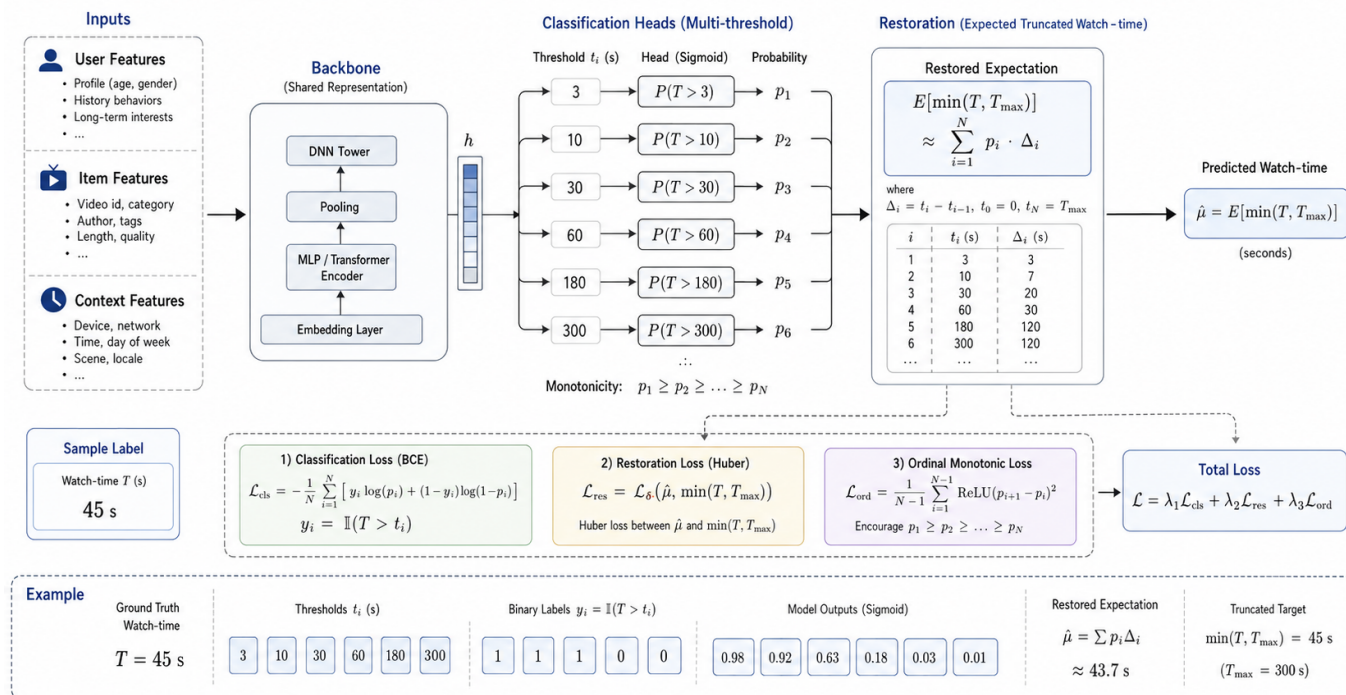
这个方案很优雅，适合短视频。直播场景不一定适合第一版采用，因为直播没有固定视频长度，状态实时变化更强，指数-高斯混合容易带来收敛和解释成本。

四、CREAD 的核心思想



图片加载失败

Watch-time Prediction with CREAD-like Classification-Restoration



读图说明： 这张图对应 CREAD-like 主流程。输入特征经过 backbone 后，不直接输出一个连续时长，而是输出多个生存概率头： $P(T > 3s)$ 、 $P(T > 10s)$ 、 $P(T > 30s)$ 等。右侧 restoration 模块用 $E[\min(T, T_{\max})] \approx \sum P(T > t_i) \Delta_i$ 把多个分类头还原成连续预测时长。下方三个 loss 分别对应多头 BCE 分类 loss、Huber 还原 loss 和 ordinal 单调保序 loss。

CREAD 全名是：

Classification-Restoration Framework with Error Adaptive Discretization

它解决的问题是：观看时长是连续值，但直接回归很难；如果离散成桶，又会出现桶怎么切、怎么还原的问题。

CREAD 的主流程是：

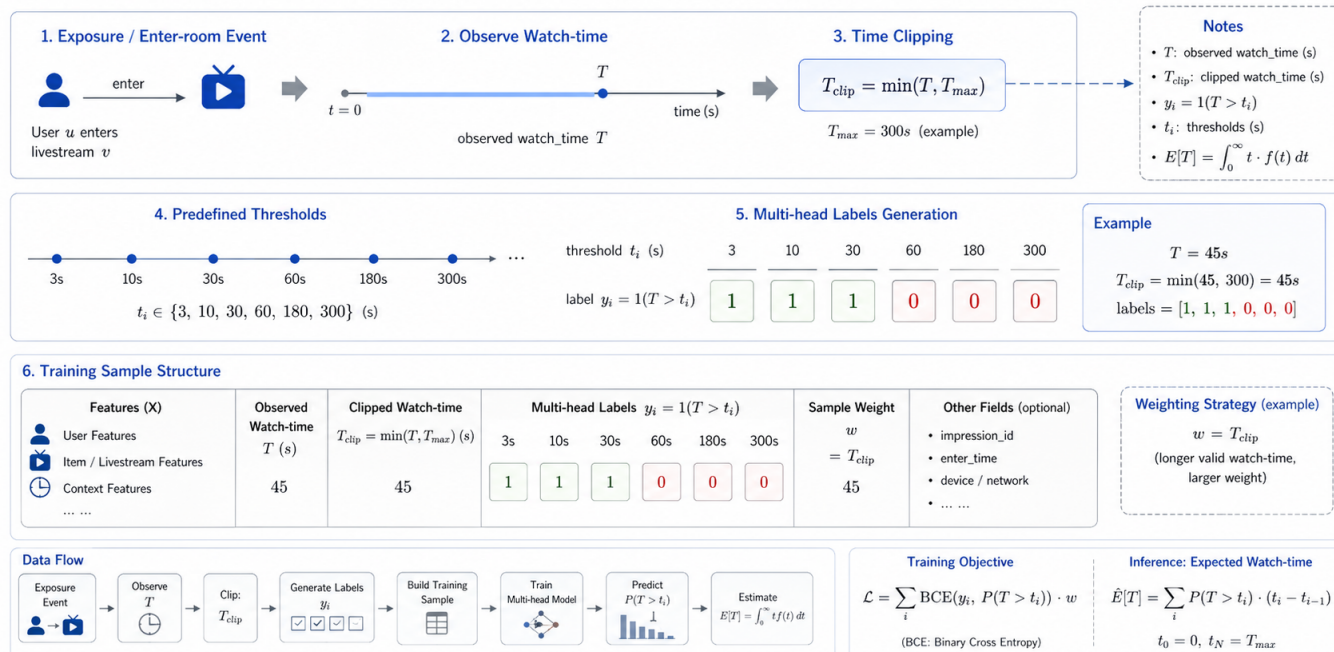
- 1 连续观看时长
- 2 => 分桶
- 3 => 多个分类任务
- 4 => 还原成连续时长
- 5 => 用分类 loss、还原 loss、保序 loss 联合训练

五、样本怎么设计



图片加载失败

Watch-time Sample Construction for Livestream Recommendation



读图说明： 图中展示了一条样本如何展开。曝光或进房后得到真实观看时长 T ，先做截断 $T_{clip} = \min(T, T_{max})$ ，再根据阈值生成多个 label: $y_i = 1(T > threshold_i)$ 。例子里 $T = 45s$ 对应 $[1, 1, 1, 0, 0, 0]$ ，因为它超过 3、10、30 秒，但没有超过 60、180、300 秒。训练时连续 label T_{clip} 和多头 label 都要保留。

假设一条样本真实观看时长是：

1 $T = 45s$

阈值是：

1 $3s, 10s, 30s, 60s, 180s, 300s$

那么这条样本会生成多个二分类 label：

1 $T > 3s \Rightarrow 1$
 2 $T > 10s \Rightarrow 1$
 3 $T > 30s \Rightarrow 1$
 4 $T > 60s \Rightarrow 0$
 5 $T > 180s \Rightarrow 0$
 6 $T > 300s \Rightarrow 0$

所以 label 向量是：

```
1  [1, 1, 1, 0, 0, 0]
```

但原始连续时长 `T=45s` 仍然保留，用于还原 loss。

也就是说，一条训练样本至少包含：

```
1  features
2  watch_time
3  watch_time_clip
4  label_gt_3
5  label_gt_10
6  label_gt_30
7  label_gt_60
8  label_gt_180
9  label_gt_300
```

样本生成伪代码：

```
1  thresholds = [3, 10, 30, 60, 180, 300]
2  Tmax = 600
3
4  watch_time_clip = min(watch_time, Tmax)
5  labels = [(watch_time_clip > t) for t in thresholds]
```

对于直播场景，建议使用截断时长：

```
1  E[min(T, Tmax)]
```

因为直播可能有人挂很久，极端长时长会显著影响训练稳定性。第一版可以取：

```
1  Tmax = 600s 或 1800s
```

取决于你的业务是更关注 10 分钟内有效观看，还是更关注 30 分钟深度观看。

六、模型怎么设计

最小可落地结构是：

```

1  用户特征、直播间特征、上下文特征、实时特征
2      ↓
3  Embedding
4      ↓
5  Backbone: DNN / DCN / DIN / MMoE / PLE
6      ↓
7  shared representation h
8      ↓
9  多个 sigmoid heads
10     ↓
11  P(T>3), P(T>10), P(T>30), ...
12     ↓
13  恢复模块
14     ↓
15  pred_watch_time

```

伪代码：

```

1  h = backbone(features)
2  logits = duration_head(h)          # [batch_size, num_thresholds]
3  probs = torch.sigmoid(logits)      # P(T > threshold_i)
4
5  pred_time = restore_duration(probs, boundaries)

```

如果你的主模型已经是多目标模型，可以把时长预估作为一个任务接入：

```

1  shared backbone
2      |— CTR tower
3      |— 互动 tower
4      |— 关注 tower
5      |— Duration tower: 多个 P(T>x) heads

```

不建议第一版把时长头做得过深。先验证目标收益，再考虑单独的时长专家网络、长时长专家、实时塔等增强。

七、分类怎么还原成连续时长

关键公式是：

$$1 \quad E[T] = \int_0^{\infty} P(T > t) dt$$

也就是说，期望时长等于生存函数下面的面积。

模型预测的是几个离散点：

```
1 P(T > 3)
2 P(T > 10)
3 P(T > 30)
4 P(T > 60)
5 P(T > 180)
6 P(T > 300)
```

所以用分段面积近似积分。

设边界是：

```
1 boundaries = [0, 3, 10, 30, 60, 180, 300, 600]
```

区间宽度是：

```
1 [3, 7, 20, 30, 120, 120, 300]
```

如果模型输出：

```
1 P(T > 3)    = 0.85
2 P(T > 10)   = 0.60
3 P(T > 30)   = 0.35
4 P(T > 60)   = 0.12
5 P(T > 180)  = 0.04
6 P(T > 300)  = 0.01
```

那么：

```
1 pred_time =
2     1.00 * 3
3     + 0.85 * 7
4     + 0.60 * 20
5     + 0.35 * 30
6     + 0.12 * 120
7     + 0.04 * 120
8     + 0.01 * 300
```

这里的权重不是模型学出来的，而是每个区间的长度：

```
1 interval_i = boundary_{i+1} - boundary_i
```

这就是“加权融合”。

实现代码：

```

1  def restore_duration(probs, boundaries):
2      # boundaries: [0, 3, 10, 30, 60, 180, 300, 600]
3      widths = torch.tensor(
4          [boundaries[i + 1] - boundaries[i] for i in range(len(boundaries)
5              - 1)],
6          device=probs.device,
7          dtype=probs.dtype,
8      )
9      # S(0) = 1, 后面接模型预测的 P(T > boundary_i)
10     survival = torch.cat([
11         torch.ones_like(probs[:, :1]),
12         probs,
13     ], dim=1)
14
15     return (survival * widths).sum(dim=1)

```

八、三个 loss 分别是什么

CREAD-like 训练通常包含三部分。

1. 分类 loss

每个阈值是一个二分类任务：

```

1  y_i = 1(T > threshold_i)
2  p_i = P(T > threshold_i)

```

分类 loss：

```

1  L_cls =  $\sum$  BCE(p_i, y_i)

```

作用是让每个头学会是否超过某个时长阈值。

代码：

```

1  cls_loss = F.binary_cross_entropy_with_logits(
2      logits,
3      labels.float(),
4      reduction="none",
5  )
6  cls_loss = (cls_loss * head_weights).mean()

```

2. 还原 loss

通过多个头恢复连续时长：

```
1 pred_time =  $\sum P(T > t_i) * interval_i$ 
```

然后和真实时长比较：

```
1 L_restore = Huber(pred_time, watch_time_clip)
```

Huber 的好处是：误差小时像 MSE，误差大时像 MAE。观看时长长尾严重，Huber 比纯 MSE 稳定。

代码：

```
1 restore_loss = F.huber_loss(  
2     pred_time,  
3     watch_time_clip.float(),  
4     delta=30.0,  
5 )
```

3. 保序 loss

如果头定义为 $P(T > x)$ ，就必须满足：

```
1 P(T > 3s) >= P(T > 10s) >= P(T > 30s) >= ...
```

因为事件之间有包含关系：

```
1 T > 60s 一定意味着 T > 30s
```

如果模型输出：

```
1 P(T > 60s) > P(T > 30s)
```

就是逻辑不一致。

保序 loss 可以写成：

```
1 L_ord =  $\sum \max(p_{i+1} - p_i, 0)$ 
```

代码：

```
1 ord_loss = torch.relu(probs[:, 1:] - probs[:, :-1]).mean()
```

总 loss:

1

$$L = L_{\text{cls}} + \alpha * L_{\text{restore}} + \beta * L_{\text{ord}}$$

第一版可以从:

1

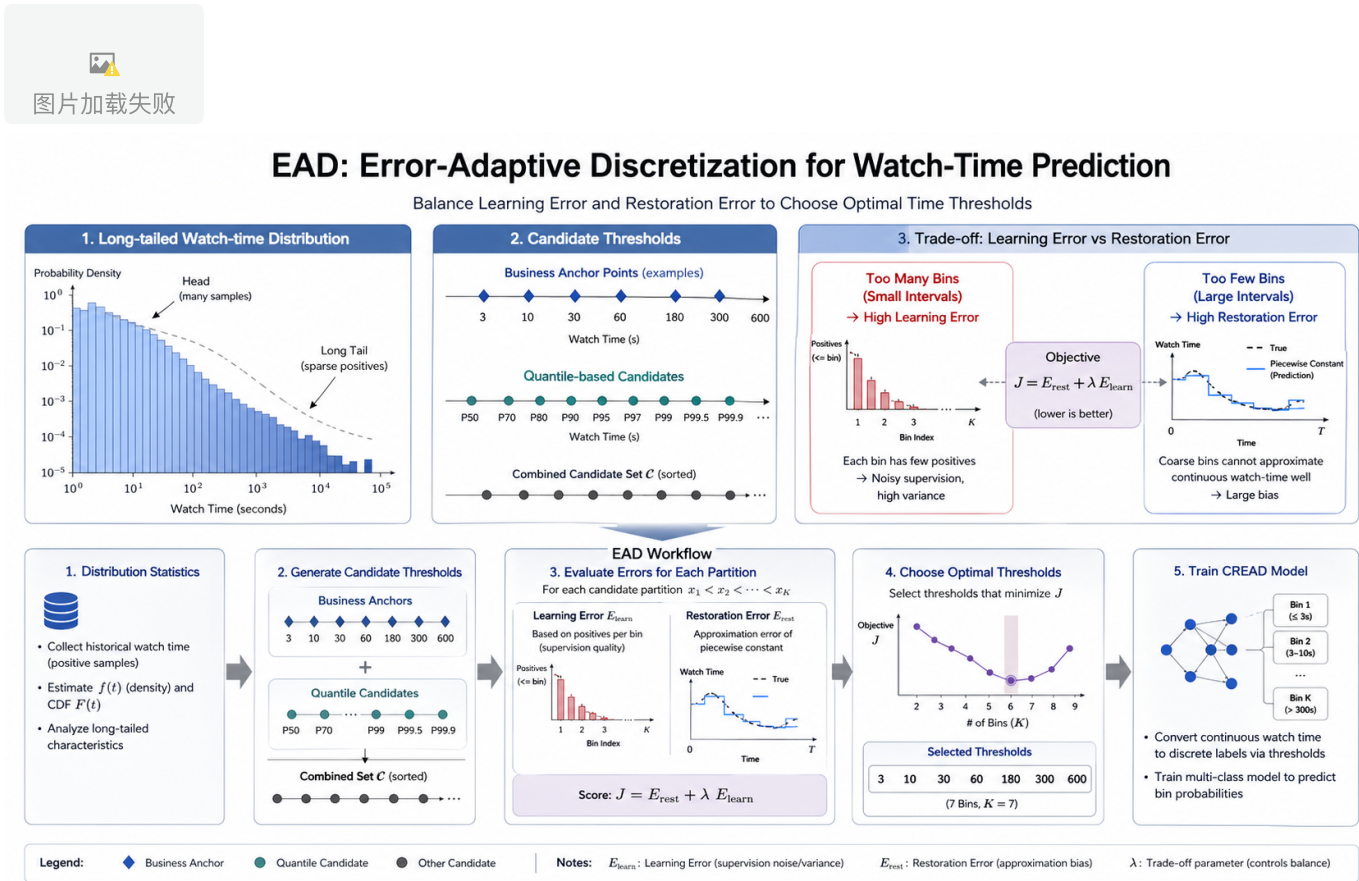
$$\alpha = 0.05 \sim 0.2$$

2

$$\beta = 0.005 \sim 0.05$$

开始调参。

九、EAD 到底怎么做



- 1 每个头正样本少
- 2 分类任务难学
- 3 learning error 大

如果阈值太少、桶太粗：

- 1 还原连续时长很粗糙
- 2 不同真实时长被压到同一个大区间
- 3 restoration error 大

EAD 的核心就是在这两个误差之间找平衡。

论文里的 EAD 会做理论分析。工程上可以先做一个近似可落地版本。

Step 1: 准备训练时长分布

先拿一段代表性训练样本，得到：

- 1 $T_1, T_2, \dots, T_N$

并做截断：

- 1 $T_i = \min(T_i, T_{\max})$

Step 2: 生成候选阈值

候选阈值来自两类。

业务锚点：

- 1 3, 10, 30, 60, 180, 300, 600

分位点：

- 1 P50, P70, P80, P90, P95, P98, P99

合并去重后形成候选集合。

Step 3: 枚举候选切法

例如比较：

- 1 ▾ 方案 A: [3, 10, 30, 60, 180, 300]
- 2 ▾ 方案 B: [3, 8, 15, 30, 60, 120, 300]
- 3 ▾ 方案 C: [5, 15, 30, 90, 180, 300, 600]
- 4 ▾ 方案 D: [分位点 + 业务锚点混合]

Step 4: 计算还原误差 proxy

假设每个桶用桶内均值代表，计算：

```
1 restore_error = mean(|T - bucket_mean(T)|)
```

或者：

```
1 restore_error = mean((T - bucket_mean(T))2)
```

桶越粗，还原误差越大。

Step 5: 计算学习难度 proxy

对每个阈值计算正例率：

```
1 pos_rate_i = P(T > threshold_i)
```

如果正例率太接近 0 或 1，这个头都不好学。

可以定义：

```
1 learn_error =  $\sum 1 / \sqrt{\text{pos\_rate\_i} * (1 - \text{pos\_rate\_i}) + \text{eps}}$ 
```

正例率越极端，惩罚越大。

Step 6: 选择总分最低的阈值方案

```
1 score = restore_error +  $\lambda$  * learn_error
```

λ 控制你更重视还原精度还是分类可学性。第一版可以通过离线实验调。

十、阈值怎么选：推荐实践

我建议第一版不要完全自动化，而是业务锚点 + 数据分布结合。

直播场景可从这组开始：

1 3s, 10s, 30s, 60s, 180s, 300s, 600s

含义：

- 1 3s: 是否秒退
- 2 10s: 是否有效进入
- 3 30s: 是否初步停留
- 4 60s: 是否有效观看
- 5 180s: 是否深度观看
- 6 300s: 是否长观看
- 7 600s: 是否超长观看

然后检查每个头正例率：

- 1 $P(T > 3s)$
- 2 $P(T > 10s)$
- 3 $P(T > 30s)$
- 4 $P(T > 60s)$
- 5 $P(T > 180s)$
- 6 $P(T > 300s)$
- 7 $P(T > 600s)$

如果某个头正例率低于 0.1%，第一版可以暂时不做，或者保留但必须加权、采样、校准。

十一、为什么要保持多头单调

如果你的头定义为：

- 1 $P(T > 3s)$
- 2 $P(T > 10s)$
- 3 $P(T > 30s)$

那么它必须单调。

原因是事件有包含关系：

- 1 $T > 30s$ 一定意味着 $T > 10s$
- 2 $T > 10s$ 一定意味着 $T > 3s$

所以：

- 1 $P(T > 3s) \geq P(T > 10s) \geq P(T > 30s)$

如果你发现业务里“不单调”，通常说明你预测的不是生存概率，而是其他目标。

例如：

- 1 $P(0-3s)$
- 2 $P(3-10s)$
- 3 $P(10-30s)$

这是区间概率，不需要单调。

或者：

- 1 $P(\text{点击})$
- 2 $P(\text{互动})$
- 3 $P(\text{关注})$
- 4 $P(\text{送礼})$

这是不同业务目标，也不需要单调。

结论：

- 1 $P(T > x)$ 头必须单调
- 2 $P(T \text{ in bucket})$ 头不要求单调
- 3 多业务目标不要求单调

十二、长时长头样本稀疏怎么处理

长时长样本非常重要，但也最难学。

常见问题：

- 1 $P(T > 300s)$ 正例率低
- 2 $P(T > 600s)$ 正例率极低
- 3 长时长样本容易被短时长海量样本淹没
- 4 模型整体 MAE 看起来还行，但长时长严重低估

建议做以下处理。

1. head-level pos_weight

每个头单独按正负比例加权：

- 1 $\text{pos_weight}_i = \text{neg_count}_i / \text{pos_count}_i$

代码：

```
1  loss = F.binary_cross_entropy_with_logits(  
2      logits,  
3      labels.float(),  
4      pos_weight=pos_weights,  
5  )
```

注意 `pos_weight` 要 clip，例如：

```
1  pos_weight <= 20 或 50
```

否则训练可能不稳定。

2. 长时长样本 oversampling

提高长时长样本进入 batch 的概率。

适合离线训练，但要注意校准偏移。最好配合：

```
1  sample_weight 反校正  
2  分桶 PCOC 校准  
3  线上小流量验证
```

3. focal loss

长时长头里，可以用 focal loss 让模型更关注难样本：

```
1  FL = -(1-p)^γ log(p)
```

常用：

```
1  γ = 1 ~ 2
```

4. 两阶段训练

第一阶段：全量样本训练整体时长能力。

第二阶段：对长时长样本或长时长头做轻量微调。

适合数据量足够、长时长目标重要的场景。

5. 截断和校准

一定要定义：

```
1 Tmax
```

并监控：

```
1 长时长桶 MAE
2  $P(T > 300s)$  PCOC
3  $P(T > 600s)$  PCOC
4 长时长样本 NDCG
```

十三、训练代码骨架

下面是一个 PyTorch 风格的最小实现。

```

1 class DurationHead(nn.Module):
2     def __init__(self, hidden_dim, num_heads):
3         super().__init__()
4         self.head = nn.Linear(hidden_dim, num_heads)
5
6     def forward(self, h):
7         logits = self.head(h)
8         probs = torch.sigmoid(logits)
9         return logits, probs
10
11
12 def make_duration_labels(watch_time, thresholds, tmax):
13     watch_time_clip = torch.clamp(watch_time, max=tmax)
14     labels = torch.stack([
15         (watch_time_clip > t).float() for t in thresholds
16     ], dim=1)
17     return watch_time_clip, labels
18
19
20 def restore_duration(probs, boundaries):
21     widths = probs.new_tensor([
22         boundaries[i + 1] - boundaries[i]
23         for i in range(len(boundaries) - 1)
24     ])
25     survival = torch.cat([torch.ones_like(probs[:, :1]), probs], dim=1)
26     return (survival * widths).sum(dim=1)
27
28
29 def duration_loss(logits, probs, labels, pred_time, watch_time_clip,
30     alpha=0.1, beta=0.01, pos_weight=None):
31     cls = F.binary_cross_entropy_with_logits(
32         logits,
33         labels,
34         pos_weight=pos_weight,
35         reduction="mean",
36     )
37     restore = F.huber_loss(pred_time, watch_time_clip, delta=30.0)
38     ordinal = torch.relu(probs[:, 1:] - probs[:, :-1]).mean()
39     return cls + alpha * restore + beta * ordinal

```

十四、离线评估与线上监控



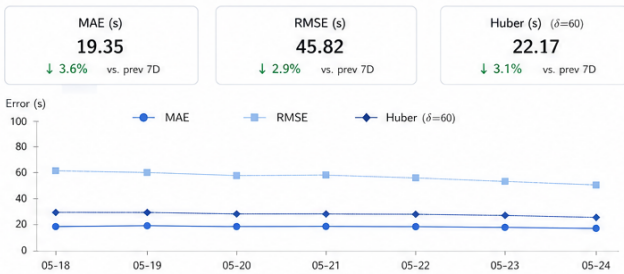
图片加载失败

CREAD Watch-time Monitoring & Calibration Dashboard

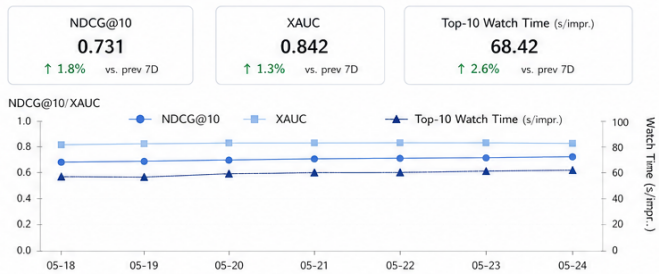
Time Range 2025-05-18 ~ 2025-05-24 Granularity Day Segment All Users

Model: CREAD-v2.3.1 Data Source: Watch Logs Traffic: 100% Impression: 25.6B Positive Watch (> 3s): 3.21B (12.5%) Max Truncation T_{max} : 600s

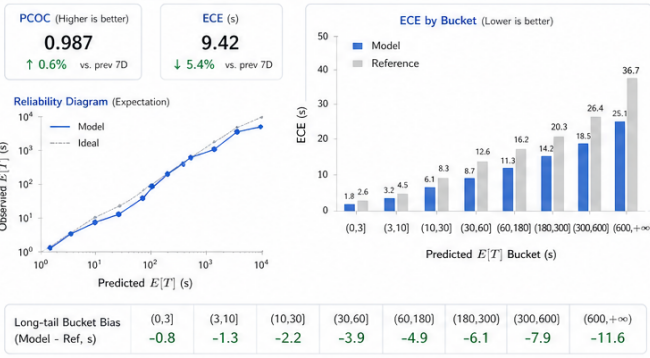
1. Error Metrics (Watch Time T in Seconds)



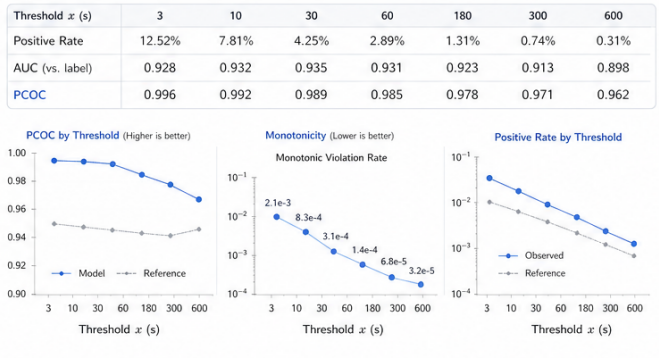
2. Ranking Metrics (Using Watch Time as Gain)



3. Calibration (Watch-time Expectation)



4. Head Health (Classification Heads $P(T > x)$)



Notes:

- Watch time is truncated at $T_{max} = 600$ s for label construction.
- PCOC: Probabilistic Concordance (higher is better).
- ECE: Expected Calibration Error in seconds.

Definitions:

- $E[T] = \int_0^{+\infty} t \cdot f(t) dt$ is the expected watch time.
- CREAD: Classification-Restoration Error-Adaptive Discretization.

Legend:

- Model
- Reference (Unbiased / Log)
- ↑ Improvement
- ↓ Degradation

读图说明： 这张图把时长预估上线后的监控拆成四类：误差指标看 MAE/RMSE/Huber，排序指标看 NDCG/XAUC/Top-K watch time，校准指标看 PCOC/ECE，多头健康度看各个 $P(T > x)$ 头的 AUC、PCOC、正例率和单调违例率。右下角的 long-tail bucket bias 用来专门监控长时长桶是否系统性低估。

时长预估不能只看 MAE。

建议同时看：

- 误差指标：MAE、RMSE、Huber
- 排序指标：NDCG、XAUC、Top-K 平均观看时长
- 校准指标：PCOC、ECE、分桶 PCOC
- 多头指标：各头 AUC、各头 PCOC、各头正例率
- 结构指标：单调违例率、pred_time 分布、长时长桶偏差
- 线上指标：有效观看、长观看、互动、关注、留存、流量分配

一个很实用的监控表：

维度	指标	目的
----	----	----

全局	MAE / RMSE	看整体误差
排序	NDCG@K	看高时长内容是否排前面
校准	PCOC	看是否系统性高估或低估
分桶	0-3s、3-10s、10-30s、30-60s、60s+	看哪个区间错
多头	P(T>x) AUC / PCOC	看每个头是否健康
单调	violation rate	看是否违反生存概率逻辑
长尾	P(T>300s)、P(T>600s)	看长时长目标是否学到

十五、推荐落地路线

第一阶段：建立 baseline

先做两个 baseline：

- 1 1. `log(1 + watch_time)` 回归
- 2 2. WLR 加权二分类

目的不是最终上线，而是建立可比较基线。

第二阶段：分桶多头

使用：

- 1 ▾ `thresholds = [3, 10, 30, 60, 180, 300]`
- 2 ▾ `boundaries = [0, 3, 10, 30, 60, 180, 300, 600]`

训练：

- 1 多头 BCE + Huber restore loss + ordinal loss

第三阶段：EAD 阈值优化

基于数据分布和业务锚点，比较多组阈值方案：

- 1 MAE
- 2 NDCG
- 3 分桶 PCOC
- 4 各头 AUC
- 5 长时长桶偏差

选择综合最优的切法。

第四阶段：长时长增强

如果长时长业务重要，加入：

- 1 pos_weight
- 2 oversampling
- 3 focal loss
- 4 长时长 head 校准
- 5 Tmax 调整

第五阶段：多目标融合

最终排序分不要只用预测时长，还要和点击、互动、关注、负反馈等目标融合：

```
1 score = f(pCTR, pDuration, pInteract, pFollow, pNegative)
```

直播里尤其要避免只追求时长导致内容生态偏移。

十六、我建议的第一版配置

如果要快速落地，我建议：

```
1 Tmax = 600s
2 thresholds = [3, 10, 30, 60, 180, 300]
3 boundaries = [0, 3, 10, 30, 60, 180, 300, 600]
4 heads = 6
5 loss = BCE + 0.1 * Huber + 0.01 * Ordinal
```

训练样本：

- 1 曝光或进房样本
- 2 保留 watch_time_clip
- 3 生成 6 个 $P(T>x)$ label
- 4 长时长头使用 clipped pos_weight

上线前必看：

- 1 各头 AUC
- 2 各头 PCOC
- 3 pred_time MAE
- 4 长时长桶偏差
- 5 单调违例率
- 6 Top-K NDCG
- 7 线上有效观看和长观看

十七、总结

时长预估的本质不是“多加一个回归头”，而是要把长尾、延迟、稀疏、偏置这些问题一起考虑。

对于直播场景，最推荐的实践方案是：

- 1 CREAD-like 分桶多头
- 2 + 生存函数还原期望时长
- 3 + Huber 还原 loss
- 4 + 单调保序 loss
- 5 + EAD 阈值选择
- 6 + 长时长头样本增强
- 7 + 分桶校准与线上监控

一句话概括：

- 1 把难学的连续观看时长，拆成多个稳定、可解释、可校准的 $P(T>x)$ 分类问题，再恢复成可用于排序的期望时长。

参考资料

- [CREAD: A Classification–Restoration Framework with Error Adaptive Discretization for Watch Time Prediction in Video Recommender Systems](#)
- [Multi–Granularity Distribution Modeling for Video Watch Time Prediction via Exponential–Gaussian Mixture Network](#)

- [Deep Neural Networks for YouTube Recommendations](#)
- [Tree based Progressive Regression Model for Watch–Time Prediction in Short–video Recommendation](#)
- [Deconfounding Duration Bias in Watch–time Prediction for Video Recommendation](#)
- [Counteracting Duration Bias in Video Recommendation via Counterfactual Watch Time](#)